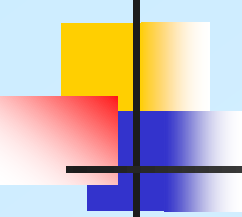


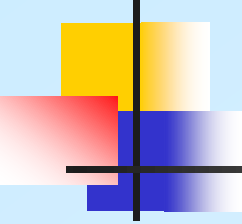
第 10 讲 编码与软件测试

Programming & Software Testing



主要内容

1. 编码的效率与稳定性
2. 测试基本概念
3. 模块测试
4. 集成测试
5. 验收测试
6. 测试方案
7. 调试
8. 系统的转换



1. 编码的效率与稳定性

- 效率：处理机时间和存储空间的使用。

- 时间复杂性：

- 多项式复杂度问题（**P**类问题 **polynomial time Turing machine**）

- 指数复杂度问题（**NP**难解性问题 **non-deterministic polynomial time Turing machine**）

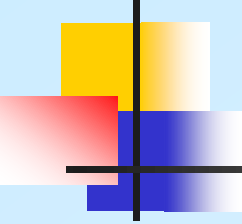
- 空间复杂性：

- 如递归的次数与堆栈溢出

1. 编码的效率与稳定性

姚期智（Andrew Chi-Chih Yao，1946年—），美籍华人，计算机科学家，研究方向**计算复杂性理论研究**，2000年图灵奖得主，是目前唯一一位获得此奖项的华人及亚洲人。清华大学理论计算机科学研究中心教授及香港中文大学博文讲座教授。



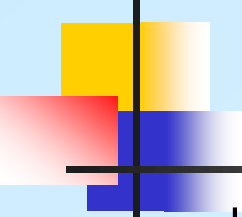


1. 编码的效率与稳定性

- 正确性不是对程序质量的唯一要求

如：梵天塔（汉诺塔）问题





1. 编码的效率与稳定性

木盘数	计 数	所需次数	乘幂式
1	1	1	2^1-1
2	$1 + 1 + 1$	3	2^2-1
3	$3 + 1 + 3$	7	2^3-1
4	$7 + 1 + 7$	15	2^4-1
5	$15 + 1 + 15$	31	2^5-1
6	$31 + 1 + 31$	63	2^6-1
7	$63 + 1 + 63$	127	2^7-1
8	$127 + 1 + 127$	255	2^8-1

$$\begin{array}{ccccccc} \dots & & \dots & & \dots & & \dots \\ 2^{64}-1 & = & 18,446,744,073,709,551,615 & & 5800\text{亿年} & (\text{1次/秒}) \\ & = & 1.8 \times 10^{19} & & & & \end{array}$$

1. 编码的效率与稳定性



国家超级计算无锡中心
National Supercomputing Center in Wuxi

<https://www.nscclwx.cn/>

中国首台全部采用国产处理器构建的世界第一的超级计算机

计算 / 加速 / 创新



“神威·太湖之光” 高效能计算系统

系统峰值性能	125.436PFlops
实测持续运算性能	93.015PFlops
处理器型号	“申威26010” 众核处理器
整机处理器个数	40960个
实整机处理器核数	10649600个
系统总内存	1310720 GB
操作系统	Raise Linux
编程语言	C、C++、Fortran
并行语言及环境	MPI、OpenMP、OpenACC等
SSD存储	230TB
在线存储	10PB, 带宽288GB/s

1. 编码的效率与稳定性

排名	1	2	3	4	5
名称	天河二号	泰坦	红杉	京	米拉
运算速度 (万亿次)	33862.7	17590	17173.2	10510	8586.61
理论峰值 (万亿次)	54902.4	27112.5	20132.7	11280.4	10066.3
功率(千瓦)	17808	8209	7890	12659.89	3945
核心数	3120000	560640	1572864	705024	786432
归属	广州国家超级计算机中心	美国能源部橡树岭国家实验室	美国劳伦斯·利弗莫尔国家实验室	日本理化研究所	美国阿尔贡国家实验室

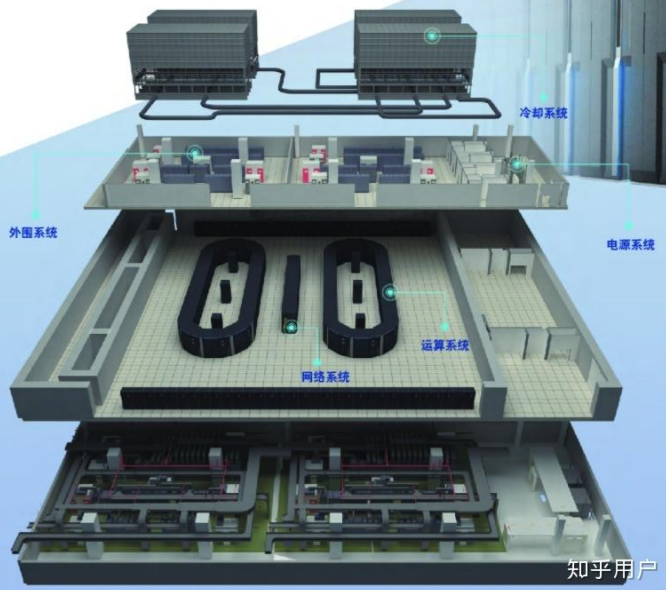


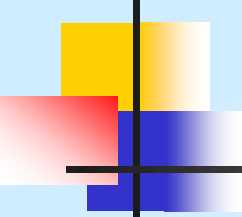
神威·太湖之光超级计算机

文/程芳

1997年，世界上首台万亿次超级计算机在美国圣地亚国家实验室落户，2016年，十亿亿次超级计算机“神威·太湖之光”问世。短短20年间，超级计算机单台性能整整提高了10万倍。“神威·太湖之光”超级计算机的最大特点就是运算速度快，其整机系统峰值性能可达每秒12.5亿亿次，它1分钟的计算能力相当于全球72亿人同时用计算器不间断计算32年。计算性能、持续性能和性能功耗比这三项关键指标在系统发布时均居世界前列。

“神威·太湖之光”超级计算机采用基于高密度弹性超节点和高流量复合网络的高效能体系结构，由运算系统、网络系统、外围系统、维护诊断系统、电源系统、冷却系统和软件系统组成。



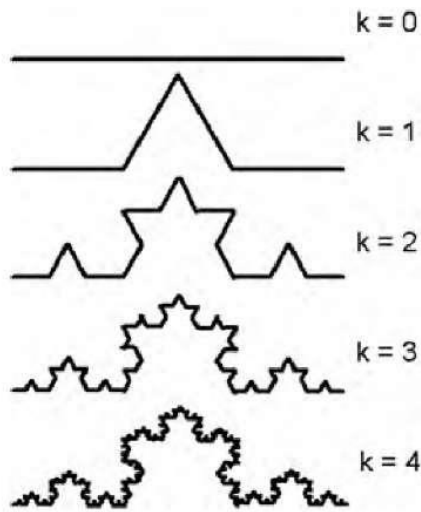


1. 编码的效率与稳定性

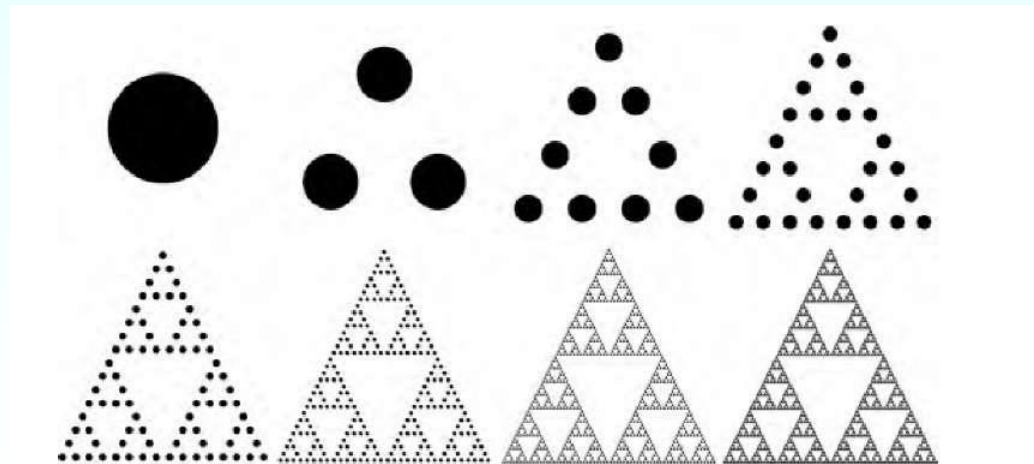
1. 太湖之光, Sunway MPP, SW26010, 国家超级计算中心, 中国无锡, 1060万个核心, 93.01pf(每秒千万亿次运算 10^{15}) $1.8 \times 10^{19} \div 93 \times 10^{15} = 3.2$ 分钟; 1481度电; 每年2.4亿度
2. 天河二号, TH-IVB-FEP Cluster, 国家超级计算机中心, 中国广州, 312万个核心, 33.86pf
3. 泰坦, Cray XK7系统, , 美国能源部, 橡树岭国家实验室, 17.59pf
4. 红杉, IBM BlueGene/Q系统, 美国能源部, 劳伦斯利弗莫尔国家实验室, 加州, 157万个核心, 16.32pf
5. Cori, Cray XC40, 伯克利实验室, 美国国家能源研究科学计算中心(NERSC), 14pf
6. Oakforest-PACs, 富士通Primergy CX1640 M1 cluster, 日本, 先进高性能计算联合中心, 13.6pf
7. K Computer, SPARC64系统, 70.5万个核心, 日本, Riken计算科学高级研究所, 10.5pf
8. Piz Daint, Cray XC30, 11.6万个Xeon和英伟达核心, 瑞士国家计算中心, 瑞士, 9.8pf
9. Mira, IBM BlueGene/Q系统, 美国DOE/SC/Argonne国家实验室, 78.6万个核心, 8.6pf
10. Trinity, Cray CX40, 美国DOE/NNSA/LANL/SNL, 30.1万个核心, 8.1pf
11. FJNU-HPCC: 736万亿次/秒=0.736pf, 544分钟, 1574元, 150万元/年

1. Dimension of the space

- Spatial Entity: 0, 1, 2, 3
- Spatial Phenomena: 2+T, 3+T
- Self-Similarity and The Fractal Dimension: **X.X**



Iterative steps to generate a
Koch curve 科赫曲线



Iterative steps to construct the
Sierpinsky gasket 谢尔平斯基镂垫



1. 编码的效率与稳定性

稳定性的保障

制度纪律

- 编码规范、代码提交门禁
- Code Review
- 静态代码扫描, 动态代码分析
- Unit Test、压测
- 灰度发布、Rollback、应急预案
- 监控
- 复盘、故障树分析

思想之道

- 保持简单、降低复杂度
- 不(零)信任、面向失败设计

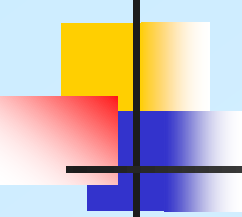
实践之术

- 冗余设计(数据、计算、带宽冗余)
- 快速恢复设计(无状态设计)
- 容错、灾备
- 隔离
- 过载保护(限流、熔断、有损服务)
- 错误重试策略, 避免流量风暴
- 去关键路径、去中心化、避免单点故障
- 负载均衡(load balance)
- 看门狗设计
- 安全编码



导致多起空难的波音**737**致命方向舵故障始末

第4章 方向舵的故事.....	294
4.1 事故情况.....	294
4.2 动力控制组件(PCU).....	307
4.3 偏航阻尼器.....	307
4.4 方向舵减压器(RPR, NG未安装).....	308
4.5 数字式偏航阻尼耦合器.....	308
4.6 方向舵系统改进项目(RSEP).....	309
4.7 压差监控器(FFM).....	309
4.8 方向舵限压器(RPL, NG未安装).....	310
4.9 载荷限制器(仅NG).....	310
4.10 QRH程序分析.....	310



1. 编码的效率与稳定性

- 编码的原则：可读性

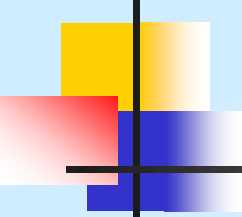
- 简明清楚，避免过分技巧，以免造成程序的可读性差。
- 避免及少用全局变量，避免副作用，不要使用过多的程序嵌套结构。
- 首先达到正确性，然后考虑执行速度。
- 采用记忆名，充分的注释。

1. 编码的效率与稳定性

- ❑ 匈牙利命名法：一种编程时的命名规范。变量名=属性+类型+对象描述，对象的名称都要求有明确含义，可以取对象名字全称或名字的一部分。命名要基于容易记忆容易理解的原则。
- ❑ 匈牙利程序员**Charles Simonyi** 发明，他在微软呆了几年，于是这种命名法就通过微软的各种产品和文档资料向世界传播开了。现在，大部分程序员不管自己使用什么软件进行开发，或多或少都使用了这种命名法。



Charles Simonyi

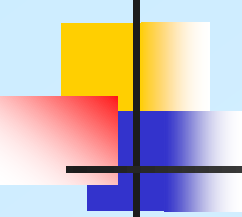


1. 编码的效率与稳定性

- 程序的可读性

实践表明，一个软件产品完成开发工作，投入运行以后，如果发生了问题，很难依靠原来的开发人员来解决：人员流动、遗忘。

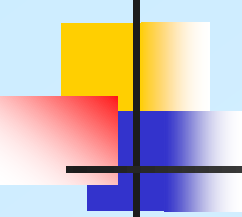
- 增加注释



1. 编码的效率与稳定性

奇异码： 很多年(15年以上)之前开发的程序

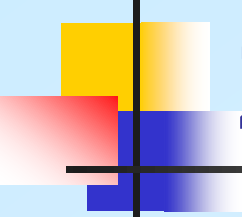
- 现有维护人员没有参与开发
- 不具备现有的开发规范
- 文档不完整,修改记录简略



1. 编码的效率与稳定性

程序注释

- 序言性注释：位于每个模块的起始部分
 - 说明每个模块的用途、功能；
 - 模块的接口、调用形式、参数描述及从属模块清单；
 - 数据描述，重要数据的名称、用途、约束等；
 - 开发历史，设计者、审阅者、日期、修改说明
- 功能性注释：源程序语句之间
 - 对程序段进行注释



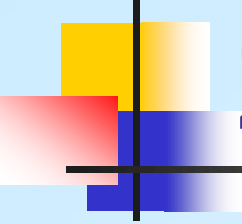
2. 测试基本概念

2.1 什么是测试

- 测试是假定程序中存在错误，因而想通过测试来发现尽可能多的错误。
- 对软件是否满足预定的需求进行验证，识别软件的实际运行结果与期望值之间的差异，而对系统或系统部件进行评价的过程（IEEE，1983）。

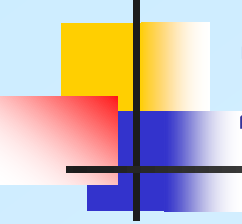
2.2 测试的目标

- 测试是为了发现程序中的错误而执行程序的过程；
- 好的测试方案是尽可能发现迄今为止尚未发现的错误的测试方案；
- 成功的测试是发现了迄今为止尚未发现的错误。



2. 测试基本概念

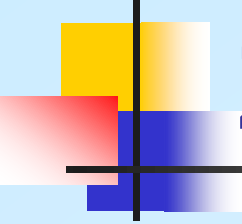
- “测试只能证明软件是有错的，但不能证明软件是没有错误的（Testing reveals the presence, but not the absence of bugs）”——戴克斯特拉
- 测试的困难：遍历程序的每一种运行状态几乎是不可能的！——“组合数爆炸”
- Traveling Salesman Problem (TSP)问题: $(n-1)!$



2. 测试基本概念

2.3 测试的原则

- 不应测试自己开发的程序
- 设计测试用例时，不仅有确定的输入数据，还有确定的输出数据
- 测试用例不仅有合理的，也要有非合理的
- 除了检查程序是否做完了它应该做的事，还要检查它是否做了不应该做的事。
- 保留全部测试用例，作为软件的组成部分
- 程序中存在错误的概率与在该段程序中已发现的错误数成正比



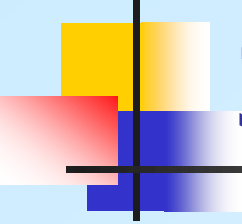
2. 测试基本概念

2.4 测试的步骤

- 模块测试
- 子系统测试
- 系统测试
- 验收测试
- 平行测试

2.5 测试过程的信息流

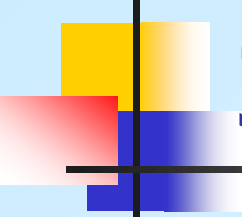
- 跟踪测试过程中语句流程。



3. 模块测试

3.1 模块测试内容

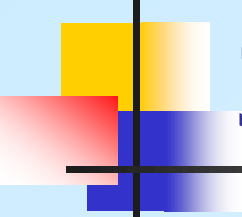
- 模块接口
- 局部数据结构
- 重要的执行通路
- 出错处理通路
- 影响上述各方面特性的边界条件



3. 模块测试

3.2 模块接口测试要点

- 参数数目和由调用模块送来的变元的数目是否相等？
- 参数的属性和变元的属性是否匹配？
- 参数和变元的单位系统是否匹配？
- 传送给被调用模块的变元的数目是否等于那个模块的参数的数目？
- 传送给被调用模块的变元属性和参数的属性是否一致？
- 传送给被调用模块的变元的单位系统和该模块参数的单位系统是否一致？
- 传送给内部函数的变元属性、数目和次序是否正确？
- 是否修改了只做输入用的变元。
- 全程变量的定义和用法在各个模块中是否一致？



3. 模块测试

3.3 测试方案中的常见错误

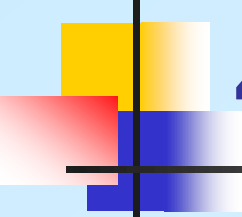
- 比较数据类型不同的量
- 逻辑运算符不正确或优先次序的错误
- 由于精度问题导致两个量不相等时，程序中却期待着相等的出现
- “差1”错：多循环一次或少循环一次
- 错误的或不存在的循环终止条件
- 当遇到发散的迭代时不能终止循环
- 错误地修改循环变量

3. 模块测试

3.4 测试软件

- 接收测试数据，把这些数据传送给被测试的模块，管理测试结果
- <https://pingcode.com/>

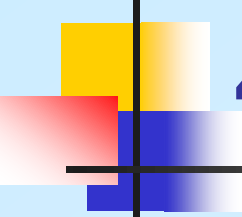




4. 集成测试

集成测试主要有两种方法

- **非渐增式测试**：先分别测试每个模块，再把所有模块按设计要求放在一起结合成所要的程序。
- **渐增式测试**：把下一个要测试的模块同已经测试好的模块结合起来进行测试，测试完以后再把下一个应该测试的模块结合进来测试。

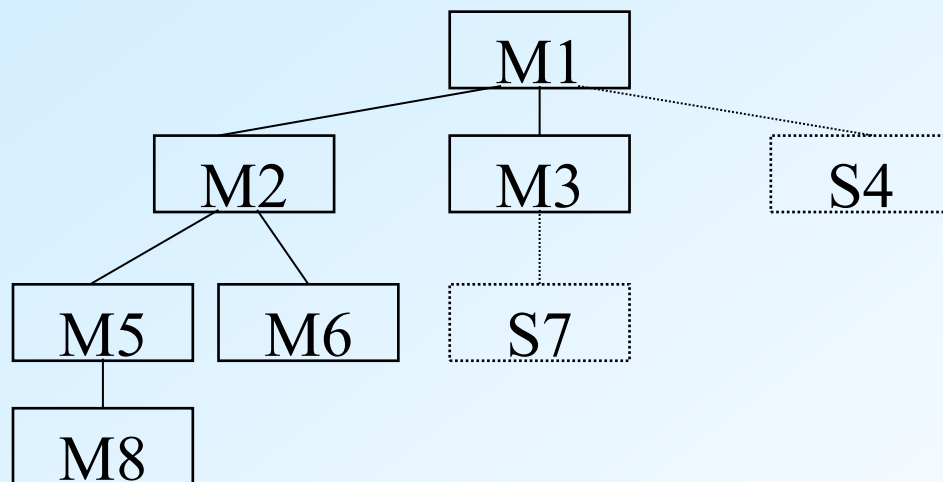


4. 集成测试

■ 在实际测试中，应该将两种方法有机结合：

- 非渐增式测试方法需要编写的软件较多，工作量较大；渐增式测试方法开销小。
- 渐增式测试方法发现模块间接口错误早；而非渐增式测试方法晚。
- 非渐增式测试方法发现错误，较难诊断；而使用渐增式测试方法，如果发生错误则往往和最近加进来的那个模块有关。
- 渐增式测试方法测试更彻底
- 渐增式测试方法需要较多的机器时间
- 使用非渐增式测试方法，可以并行测试。

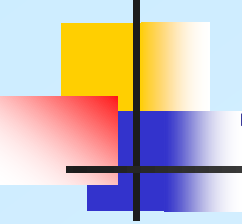
4. 集成测试



深度优先: M1->M2->M5->M8->M6->M3->S7->S4

宽度优先: M1->M2->M3->S4->M5->M6->S7->M8

比较: 发现错误的早晚?



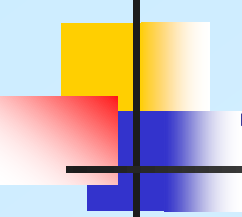
5. 验收测试

□ 目的:

- 向未来的用户表明系统能够象预定要求（需求分析）那样工作。

□ 验收测试的范围

- 验收测试，以《需求分析报告》为主要依据；
- 验收测试必须有用户参与，甚至以用户为主；
- 验收测试一般使用黑盒测试法；



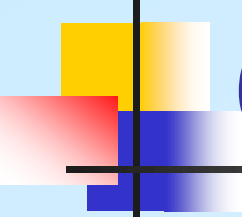
5. 验收测试

□ 验收测试的结果

- 功能和性能与用户要求一致，软件是可以接受的。
- 功能或性能与用户的要求有差距。
- 在这个阶段发现问题往往与“需求分析”阶段的差错有关，涉及的面比较广，难以解决。

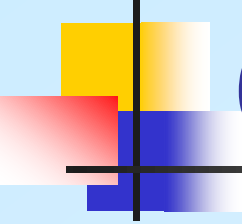
□ 复查软件配置

- 保证软件配置的所有成分都齐全，各方面的质量都符合要求，文档与程序一致。
- 除合同规定的内容和要求外，一般严格遵循用户指南以及其它操作程序。



6. 测试方案/测试用例设计

- 设计测试方案是测试阶段的关键技术问题。
- 测试方案（测试用例**Test Case**）包括要测试的功能，应该输入的测试数据和预期的结果。
- 设计技术**主要有两种方法：黑盒法与白盒法**。一般用黑盒法设计基本的测试方案，再用白盒法补充一些方案。



6. 测试方案

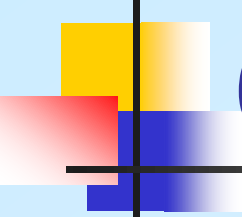
6.1 黑盒法

- 测试人员将程序看成是一个“黑盒”，即不关心程序内部是什么，只要检查程序是否符合它的“功能说明”。

(1) 等价分类法：关键在于如何将输入数据分类

- 将输入数据的可能值分成若干“等价类”，每一类以一个代表性的测试数据进行测试，这个数据就等价于这一类中的其它数据。

1~999: $x < 1$, $1 \leq x < 999$, $x \geq 999$



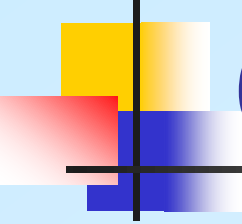
6. 测试方案

(2) 边缘值分析法：用边缘特殊值测试。

- 经验表明：程序往往在边缘情况时犯错误，故测试边缘情况比较有效。

输入数据的值的范围是：-1.0至1.0，

则可选-1.0， 1.0， -1.001， 1.001等数据作为测试数据。



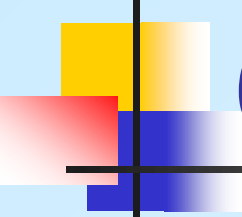
6. 测试方案

(3) 因果图法：着重分析输入条件的各种组合

- 等价类法与边缘值分析法的缺点是没有检查各种输入条件的组合。
- 每种组合条件就是“因”，它必然有一个输出的结果，这就是“果”。

(4) 错误推测法

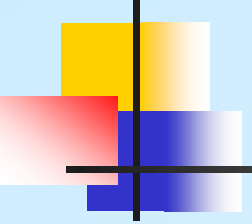
- 通过经验或直觉推测程序中可能存在的各种错误，从而有针对性设计测试用例。

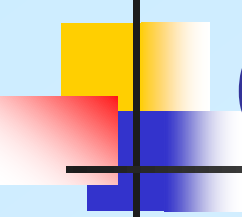


6. 测试方案

6.2 白盒法（逻辑覆盖法）

- 了解程序的功能与结构，测试用例必须根据程序内部的逻辑来设计。
- 如果想用白盒法发现程序中的所有错误，则至少必须使程序中每种可能的路径都执行一次。
- “彻底地测试”是不可能的，故策略是：在有限的时间、费用成本条件下，通过执行有限的测试用例，尽可能多地发现一些错误。
- 因此，需要根据逻辑结构设计测试用例

- 
-
- https://www.doubao.com/chat/38429909896455682?channel=gdt_sem

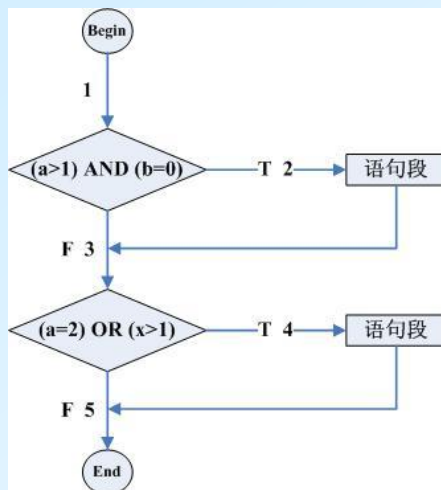


6. 测试方案

逻辑覆盖（由弱到强）：

- 语句覆盖**Statement Coverage**: 即每个语句至少能执行一次
- 判定覆盖**Decision Coverage**: 又叫分支覆盖，每个判定的分支至少执行一次
- 条件覆盖**Condition Coverage**: 判断语句中的每个条件都获得各种可能的结果
- 判断/条件覆盖**Decision/Condition Coverage**: 判断中每个条件都取得各种可能值，并使每个判断表达式也取到各种可能的结果
- 条件组合覆盖**Branch Condition Combination Coverage**: 所有判断中所有条件的各种可能组合都至少出现一次
- 路径覆盖**Path Coverage**: 所有可能的执行通路全覆盖

6. 测试方案



■ 语句覆盖：1-2-4

■ 测试用例：a=2, b=0, x=2

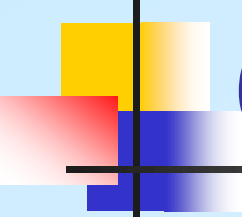
■ 漏洞：?

```
Public int Rf(int a, int b)
{
    return a / b;
}
```

Test Case:

a = 10, b = 5

漏洞：?



6. 测试方案

■判定表达式: **$a > 1 \text{ or } b > 1$**

◆用判定覆盖设计的测试数据:

- $a=5$ 判定表达式的值为“真”
- $a=0$ 判定表达式的值为“假”

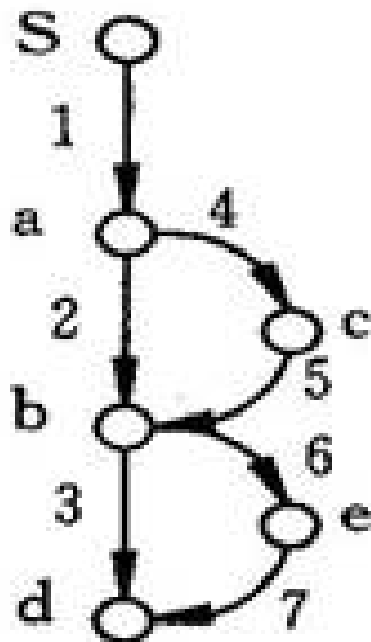
这里不需要管**b**的取值，就已经满足判定覆盖的条件了。

◆用条件覆盖设计的测试数据:

- $a=5$ 条件 $a > 1$ 的值为“真”
- $a=0$ 条件 $a > 1$ 的值为“假”
- $b=5$ 条件 $b > 1$ 的值为“真”
- $b=0$ 条件 $b > 1$ 的值为“假”

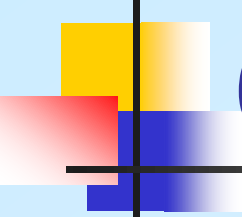
这里不考虑 $a > 1 \text{ or } b > 1$ 这个表达式的取值的情况，但必须把 $a > 1 \text{ or } b > 1$ 这两个条件的取值考虑周全。

6. 测试方案



■ 给出按照路径覆盖原则生成的覆盖路径集合

- 1-2-3
- 1-2-6-7
- 1-4-5-3
- 1-4-5-6-7



6. 测试方案

请给出解一元二次方程的python程序，必须对三个系数a, b, c进行详细的讨论。

这个程序可以解一元二次方程 $ax^2 + bx + c = 0$ ，并根据三个系数 a、b、c 的不同情况进行详细讨论：

1. 当 $a=0$ 时，方程退化为一元一次方程：

- 若 $b=0$ ，进一步检查 c：
 - 若 $c=0$ ，方程为恒等式，x 可以为任意实数
 - 若 $c \neq 0$ ，方程不成立，无解
- 若 $b \neq 0$ ，方程有唯一解 $x = -c/b$

2. 当 $a \neq 0$ 时，方程为二次方程，计算判别式 $\Delta = b^2 - 4ac$ ：

- 若 $\Delta < 0$ ，方程无实根，但有两个共轭复根
- 若 $\Delta = 0$ ，方程有两个相等的实根
- 若 $\Delta > 0$ ，方程有两个不相等的实根

程序包含完整的输入验证和异常处理，可以处理用户输入非数字的情况，并给出清晰的结果输出。

```
# 主程序
if __name__ == "__main__":
    try:
        # 获取用户输入的三个系数
        a = float(input("请输入系数 a: "))
        b = float(input("请输入系数 b: "))
        c = float(input("请输入系数 c: "))

        # 调用函数求解方程
        result = solve_quadratic(a, b, c)

        # 输出结果
        print(f"方程 {a}x² + {b}x + {c} = 0 的解为:")
        print(result)
    except ValueError:
        print("输入错误，请输入有效的数字")
    except Exception as e:
        print(f"发生错误: {e}")
```

```
1 import math
```

```
2  
3 def solve_quadratic(a, b, c):
```

```
4     """解一元二次方程  $ax^2 + bx + c = 0$ ，返回方程的根"""
```

```
5     # 检查a是否为0
```

```
6     if a == 0:
```

```
7         # a=0时，方程退化为一元一次方程  $bx + c = 0$ 
```

```
8         if b == 0:
```

```
9             if c == 0:
```

```
10                 return "方程为恒等式，x可以为任意实数"
```

```
11             else:
```

```
12                 return "方程不成立，无解"
```

```
13         else:
```

```
14             # 一次方程的解为  $x = -c/b$ 
```

```
15             x = -c / b
```

```
16             return f"一次方程的解为  $x = \{x\}$ "
```

```
17     else:
```

```
18         # 计算判别式  $\Delta = b^2 - 4ac$ 
```

```
19         delta = b**2 - 4*a*c
```

```
20  
21         # 根据判别式的值讨论方程的根
```

```
22         if delta < 0:
```

```
23             # 判别式小于0，方程无实根
```

```
24             real_part = -b / (2*a)
```

```
25             imaginary_part = math.sqrt(-delta) / (2*a)
```

```
26             return f"方程无实根，有两个共轭复根:  $x_1 = \{real\_part\} + \{imaginary\_part\}i$ ,  $x_2 = \{real\_part\} - \{imaginary\_part\}i$ "
```

```
27         elif delta == 0:
```

```
28             # 判别式等于0，方程有两个相等的实根
```

```
29             x = -b / (2*a)
```

```
30             return f"方程有两个相等的实根:  $x_1 = x_2 = \{x\}$ "
```

```
31         else:
```

```
32             # 判别式大于0，方程有两个不相等的实根
```

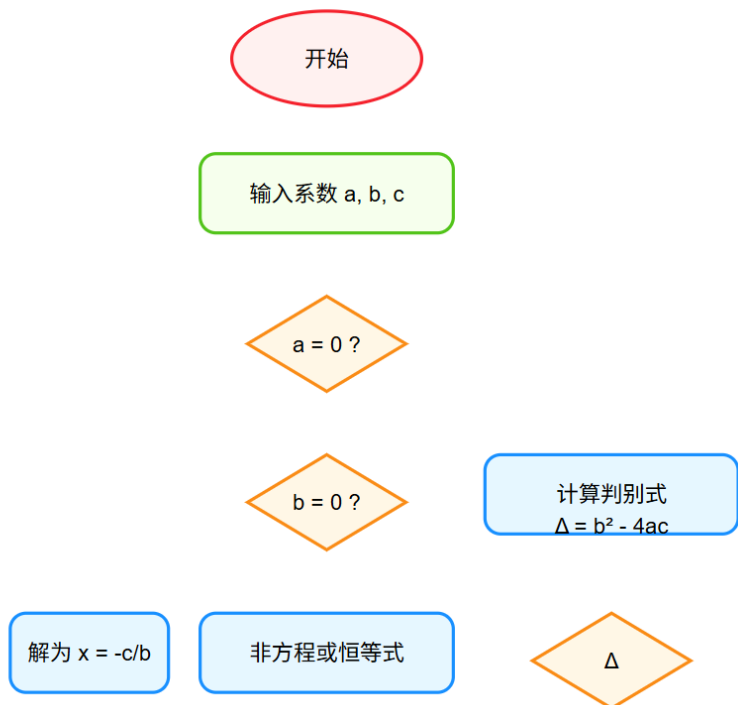
```
33             sqrt_delta = math.sqrt(delta)
```

```
34             x1 = (-b + sqrt_delta) / (2*a)
```

```
35             x2 = (-b - sqrt_delta) / (2*a)
```

```
36             return f"方程有两个不相等的实根:  $x_1 = \{x_1\}$ ,  $x_2 = \{x_2\}$ "
```

解一元二次方程 $ax^2 + bx + c = 0$ 的程序流程图



6. 测试方案

开始

├输入系数 a, b, c

├判断 a 是否为 0?

├是 → 进入一次方程分支:

├├判断 b 是否为 0?

├├├是 → 判断 c 是否为 0?

├├├├是 → 输出 "无穷多解 (全体实数)"

├├├├否 → 输出 "无解"

├├├├否 → 计算 $x = -c/b$, 输出 "唯一解: $x = \text{值}$ "

├├否 → 进入二次方程分支:

├├├计算判别式 $D = b^2 - 4ac$

├├├判断 D 的符号:

├├├├ $D > 0$ → 计算实根 $x_1, x_2 = [-b \pm \sqrt{D}]/(2a)$, 输出 "两实根: x_1, x_2 "

├├├├ $D = 0$ → 计算重根 $x = -b/(2a)$, 输出 "重根: $x = \text{值}$ "

├├├├ $D < 0$ → 计算虚根: 实部 = $-b/(2a)$, 虚部 = $\pm\sqrt{-D}/(2a)$, 输出 "共轭虚根: 实部 $\pm$ 虚部 i "

结束

解一元二次方程 $ax^2 + bx + c = 0$ 的NS图

开始

输入系数 a, b, c

$a = 0 ?$

是

否

$b = 0 ?$

是

非方程或恒等式

计算判别式
 $\Delta = b^2 - 4ac$

Δ

解

6. 测试方案

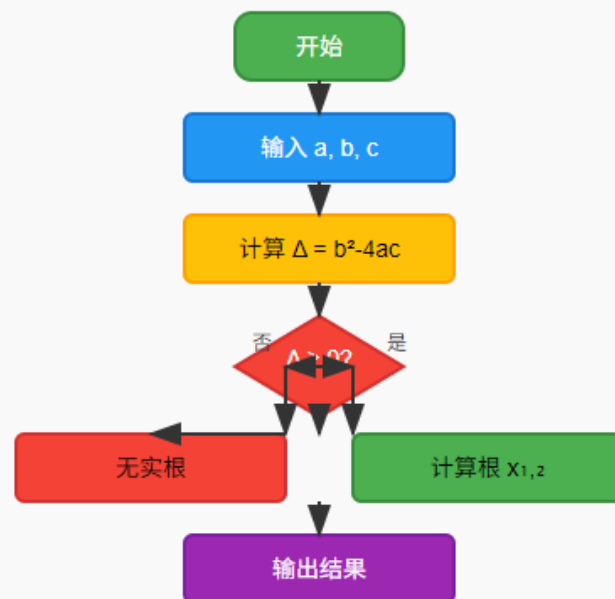
- 豆包：对这段程序，请分别给出一个语句覆盖、判定覆盖、条件覆盖、判定与条件都覆盖、条件组合覆盖、路径覆盖的测试用例。

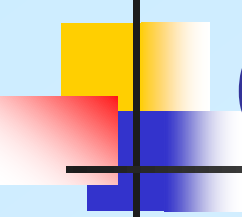
测试说明

- 语句覆盖：确保每个代码语句至少执行一次
- 判定覆盖：确保每个判定的真假分支至少执行一次
- 条件覆盖：确保每个条件的可能取值至少执行一次
- 判定 - 条件覆盖：同时满足判定覆盖和条件覆盖
- 条件组合覆盖：确保每个判定中条件的所有可能组合至少执行一次
- 路径覆盖：确保程序中的每条可能路径至少执行一次

以上测试用例可以全面覆盖程序的各种逻辑分支，保证代码质量。

解一元二次方程 NS图





6. 测试方案

1. 语句覆盖测试用例

语句覆盖要求程序中的每个语句至少被执行一次。

python ^

运行 复制 撤销 重做 全屏

测试用例1: 语句覆盖

```
def test_statement_coverage():
```

```
    # a≠0, Δ>0, 执行所有二次方程实根分支
```

```
    a, b, c = 1, -3, 2 # Δ = 9-8=1 >0
```

```
    result = solve_quadratic(a, b, c)
```

```
    assert "两个不相等的实根" in result, "测试失败"
```

```
    # a=0, b≠0, 执行一次方程分支
```

```
    a, b, c = 0, 2, -4
```

```
    result = solve_quadratic(a, b, c)
```

```
    assert "一次方程的解" in result, "测试失败"
```

2. 判定覆盖测试用例

判定覆盖要求每个判定的取真分支和取假分支至少被执行一次。

python ^

运行 复制 撤销 重做 全屏

```
# 测试用例2: 判定覆盖
def test_decision_coverage():
    # a≠0分支 ( $\Delta > 0$ ,  $\Delta = 0$ ,  $\Delta < 0$ )
    a, b, c = 1, -3, 2 #  $\Delta = 1 > 0$ 
    result = solve_quadratic(a, b, c)
    assert "两个不相等的实根" in result

    a, b, c = 1, -2, 1 #  $\Delta = 0$ 
    result = solve_quadratic(a, b, c)
    assert "两个相等的实根" in result

    a, b, c = 1, 0, 1 #  $\Delta = -4 < 0$ 
    result = solve_quadratic(a, b, c)
    assert "无实根" in result

    # a=0分支 (b≠0, b=0)
    a, b, c = 0, 2, -4 # b≠0
    result = solve_quadratic(a, b, c)
    assert "一次方程的解" in result

    a, b, c = 0, 0, 5 # b=0, c≠0
    result = solve_quadratic(a, b, c)
    assert "无解" in result
```

3. 条件覆盖测试用例

条件覆盖要求每个判定中的每个条件的可能取值至少被执行一次。

python ^

运行 复制 撤销 重做 全屏

测试用例3: 条件覆盖

```
def test_condition_coverage():
```

```
    # 覆盖 $a=0$ ,  $a \neq 0$ 
```

```
    a, b, c = 0, 5, 3
```

```
    result = solve_quadratic(a, b, c)
```

```
    assert "一次方程的解" in result
```

```
    a, b, c = 2, 4, 1 #  $a \neq 0$ ,  $\Delta=8>0$ 
```

```
    result = solve_quadratic(a, b, c)
```

```
    assert "两个不相等的实根" in result
```

```
    # 覆盖 $b=0$ ,  $b \neq 0$  (当 $a=0$ 时)
```

```
    a, b, c = 0, 0, 5
```

```
    result = solve_quadratic(a, b, c)
```

```
    assert "无解" in result
```

```
    # 覆盖 $c=0$ ,  $c \neq 0$  (当 $a=0, b=0$ 时)
```

```
    a, b, c = 0, 0, 0
```

```
    result = solve_quadratic(a, b, c)
```

```
    assert "任意实数" in result
```

```
    # 覆盖 $\Delta < 0$ ,  $\Delta \geq 0$  (当 $a \neq 0$ 时)
```

```
    a, b, c = 1, 0, 1 #  $\Delta=-4<0$ 
```

```
    result = solve_quadratic(a, b, c)
```

```
    assert "无实根" in result
```

4. 判定 - 条件覆盖测试用例

判定 - 条件覆盖要求同时满足判定覆盖和条件覆盖。

python ^

运行 复制 撤销 重做 全屏

测试用例4: 判定-条件覆盖

```
def test_decision_condition_coverage():
```

```
    # a=0分支及子条件
```

```
    a, b, c = 0, 5, 3 # a=0, b≠0
```

```
    result = solve_quadratic(a, b, c)
```

```
    assert "一次方程的解" in result
```

```
    a, b, c = 0, 0, 5 # a=0, b=0, c≠0
```

```
    result = solve_quadratic(a, b, c)
```

```
    assert "无解" in result
```

```
    a, b, c = 0, 0, 0 # a=0, b=0, c=0
```

```
    result = solve_quadratic(a, b, c)
```

```
    assert "任意实数" in result
```

```
    # a≠0分支及子条件
```

```
    a, b, c = 1, 0, 1 # a≠0,  $\Delta < 0$ 
```

```
    result = solve_quadratic(a, b, c)
```

```
    assert "无实根" in result
```

```
    a, b, c = 1, -2, 1 # a≠0,  $\Delta = 0$ 
```

```
    result = solve_quadratic(a, b, c)
```

```
    assert "两个相等的实根" in result
```

```
    a, b, c = 1, -3, 2 # a≠0,  $\Delta > 0$ 
```

```
    result = solve_quadratic(a, b, c)
```

```
    assert "两个不相等的实根" in result
```



条件组合覆盖要求每个判定中条件的各种可能组合都至少被执行一次。

python ^

运行 复制 撤销 重做 全屏

测试用例5: 条件组合覆盖

```
def test_condition_combination_coverage():
```

```
    # 组合1: a=0, b=0, c=0
```

```
    a, b, c = 0, 0, 0
```

```
    result = solve_quadratic(a, b, c)
```

```
    assert "任意实数" in result
```

```
    # 组合2: a=0, b=0, c≠0
```

```
    a, b, c = 0, 0, 5
```

```
    result = solve_quadratic(a, b, c)
```

```
    assert "无解" in result
```

```
    # 组合3: a=0, b≠0
```

```
    a, b, c = 0, 5, 3
```

```
    result = solve_quadratic(a, b, c)
```

```
    assert "一次方程的解" in result
```

```
    # 组合4: a≠0,  $\Delta < 0$ 
```

```
    a, b, c = 1, 0, 1
```

```
    result = solve_quadratic(a, b, c)
```

```
    assert "无实根" in result
```

```
    # 组合5: a≠0,  $\Delta = 0$ 
```

```
    a, b, c = 1, -2, 1
```

```
    result = solve_quadratic(a, b, c)
```

```
    assert "两个相等的实根" in result
```

```
    # 组合6: a≠0,  $\Delta > 0$ 
```

```
    a, b, c = 1, -3, 2
```

```
    result = solve_quadratic(a, b, c)
```

```
    assert "两个不相等的实根" in result
```

5. 条件组合覆盖测试用例



路径覆盖要求程序中的每一条可能路径都至少被执行一次。

python ^

运行 复制 撤销 重做 全屏

测试用例6: 路径覆盖

```
def test_path_coverage():
```

```
    # 路径1: a=0, b=0, c=0
```

```
    a, b, c = 0, 0, 0
```

```
    result = solve_quadratic(a, b, c)
```

```
    assert "任意实数" in result
```

```
    # 路径2: a=0, b=0, c≠0
```

```
    a, b, c = 0, 0, 5
```

```
    result = solve_quadratic(a, b, c)
```

```
    assert "无解" in result
```

```
    # 路径3: a=0, b≠0
```

```
    a, b, c = 0, 5, 3
```

```
    result = solve_quadratic(a, b, c)
```

```
    assert "一次方程的解" in result
```

```
    # 路径4: a≠0,  $\Delta < 0$ 
```

```
    a, b, c = 1, 0, 1
```

```
    result = solve_quadratic(a, b, c)
```

```
    assert "无实根" in result
```

```
    # 路径5: a≠0,  $\Delta = 0$ 
```

```
    a, b, c = 1, -2, 1
```

```
    result = solve_quadratic(a, b, c)
```

```
    assert "两个相等的实根" in result
```

```
    # 路径6: a≠0,  $\Delta > 0$ 
```

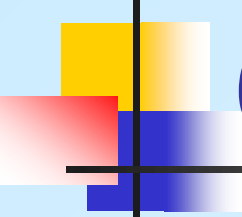
```
    a, b, c = 1, -3, 2
```

```
    result = solve_quadratic(a, b, c)
```

```
    assert "两个不相等的实根" in result
```

6. 路径覆盖测试用例

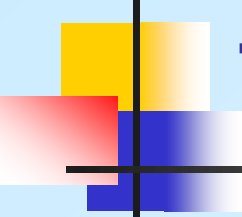




6. 测试方案

6.3 实用测试策略

- 在任何情况下都使用边界值分析的方法。
- 必要时用等价划分法补充测试方案。
- 必要时再用错误推测法补充测试方案。
- 对照程序逻辑，检查已经设计出的测试方案。可以根据对程序可靠性的要求采用不同的逻辑覆盖标准，如果现有测试方案的逻辑覆盖程度没达到要求的覆盖标准，则应再补充一些测试方案。
- 注意：即使采用综合策略设计方案，仍不能保证测试将发现一切程序错误。

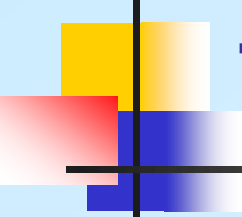


7. 调试

(1) 基本任务：诊断和改正程序中的错误

(2) 调试技术

- 输出存储器内容：以八进制或十六进制的形式输出存储器的内容。
- 打印语句：打印关键输出变量的值
- 自动工具：利用程序设计语言的调试功能或者使用专门的软件工具分析程序的动态行为。



7. 调试

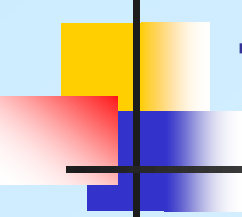
(3) 调试策略

(a) 试探法：缓慢而低效

- 调试人员分析错误征兆，猜想故障的大致位置，然后使用前述的一两种调试技术，获取程序中被怀疑的地方附近的信息。

(b) 回溯法：适用于小规模程序模块

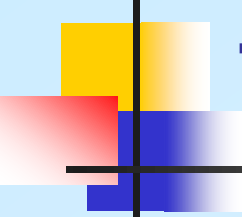
- 确定最先发现“症状”的地方，然后人工沿程序的控制流往回追踪源程序代码，直到找出错误根源或确定故障范围为止。
- 回溯法的另一种形式是正向跟踪，也就是使用输出语句检查一系列中间结果，以确定最先出现错误的地方。



7. 调试

(c) 对分查找法：比较快

- 如果已经知道每个变量在程序内若干个关键点的正确值，则可以用赋值语句或输入语句在程序中间点附近“注入”这些变量的正确值，然后检查程序的输出。如果输出结果是正确的，则故障在程序的前半部分；反之，故障在程序的后半部分。对于程序中有故障的那部分再重复使用这个方法，直到把故障范围缩小到容易诊断的程度为止。



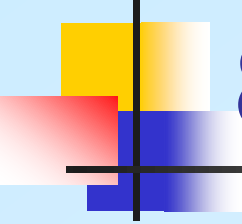
7. 调试

(d) 归纳法

- 从个别推断一般的方法。从线索（错误征兆）出发，通过分析这些线索之间的关系而找出故障。

(e) 演绎法

- 演绎法从一般原理或前提出发，经过删除和精化的过程推导出结论。用演绎法调试开始时先列出所有看来可能成立的原因或假设，然后一个一个地排除列举出的原因，最后证明剩下的原因确实是错误的根源。



8. 系统的转换与交付使用

(1) 什么是系统转换

- 由于一个组织（企业）的管理工作是连续进行的，软件系统也必须连续地进行工作，这就有一个老的软件系统与新的软件系统的交替过程，即老的软件系统逐渐地退出，由新的软件系统来代替。称为系统的转换（或切换）。

8. 系统的转换与交付使用

千年虫问题：

对闰年的计算问题，不能把2000年识别为闰年，即在该计算机系统的日历中没有2000年2月29日这一天，而是直接由2000年2月28日过渡到了2000年3月1日；

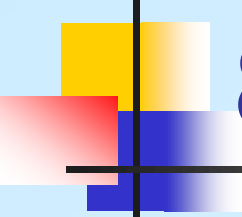
两位十进制数来表示年份：1999年12月31日进入2000年1月1日后，系统将无法正常识别由“00”表示的2000年（识别为1900年），引发各种各样的计算机业务处理系统和控制系统的功能紊乱。



8. 系统的转换与交付使用

- 北京市计算机2000年问题办公室
- 你在1999年12月31日23: 59分打了三分钟的电话，电话局的账单却可能显示为(100年—3分钟)；
- 美国联邦核管处担心全美的百余座核电厂里的仪器由于2000年问题失控造成核辐射外泄等灾难。
- 税务局的电脑可能会认为你拖欠了100年的税款，从而寄来天文数字般的补税通知。
- 由于控制雷达的电脑失灵，空中管制完全瘫痪，班机停飞。



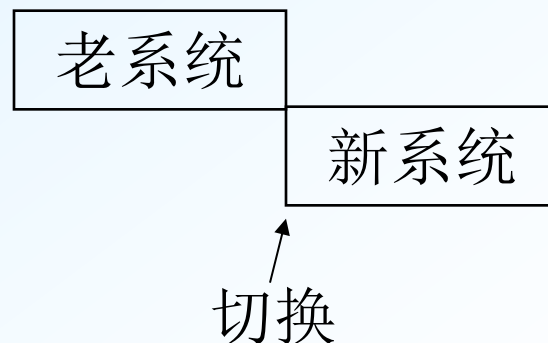


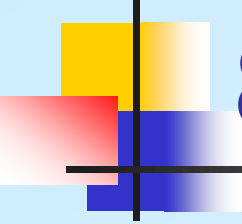
8. 系统的转换与交付使用

(2) 系统转换的方式

a) 直接转换

- 在某一时刻，老系统停止运行，新系统立即开始运行。
- 特点：简单，最省费用，但又很大风险。
- 适用范围：对一些小系统，或者新系统已进行过多次真实测试，或者老系统已完全不能使用。

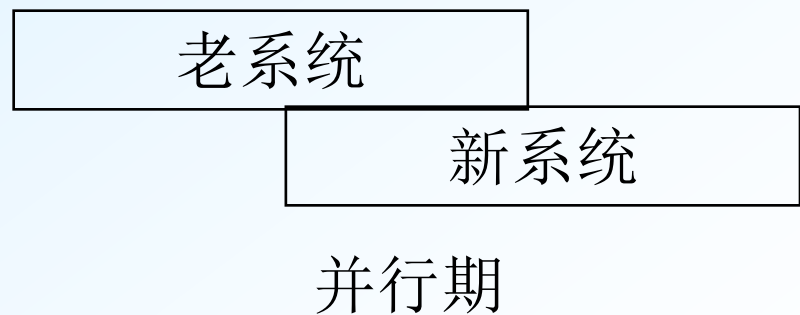




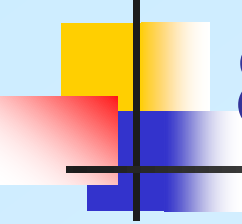
8. 系统的转换与交付使用

(b)并行切换

- 新老系统并行工作一段时间，经考验后新系统代替旧系统。
- 特点：风险小，但费用大，常用。
- 并行时间一般为3-6个月，甚至1年。



2017-2018学校财务处系统更新

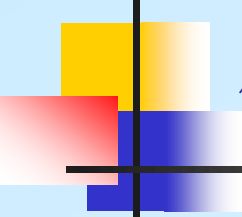


8. 系统的转换与交付使用

(3)分段切换

- 该切换方式是前两种切换方式的结合。在新系统正式全部运行前，一部分一部分代替老系统。
- 特点：既可靠，又节省费用，但要注意子系统间、功能间的接口问题。
- 一般较大系统皆采用之。





作业

- 写出课程设计中核心模块的算法，如判定表判定树，基于白盒法、黑盒法、各种逻辑覆盖，分别设计测试用例。
- 跟踪测试流程，对算法的效率进行评估